Reguläre Sprachen, Ausdrucksstärke (Teil 1)

BC George (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Motivation

Was muss ein Compiler wohl als erstes tun?

Schlüsselwörter, Bezeichner (= Namen von Variablen, Funktionen, Klassen, Methoden ...) identifizieren

Themen für heute

- Lexer = Scanner = Zerteiler
- Endliche Automaten
- Reguläre Sprachen

Endliche Automaten

Alphabete

Sigma

Def.: Ein **Alphabet** Σ ist eine endliche, nicht-leere Menge von Symbolen. Die Symbole eines Alphabets heißen *Buchstaben*.

epsilon

Def.: Ein **Wort** *w über einem Alphabet* Σ ist eine endliche Folge von Symbolen aus Σ. ϵ ist das leere Wort. Die *Länge* $|w|$ eines Wortes w ist die Anzahl von Buchstaben, die es enthält (Kardinalität).

Def.: Eine **Sprache** *L über einem Alphabet* Σ ist eine Menge von Wörtern über diesem Alphabet. Sprachen können endlich oder unendlich viele Wörter enthalten.

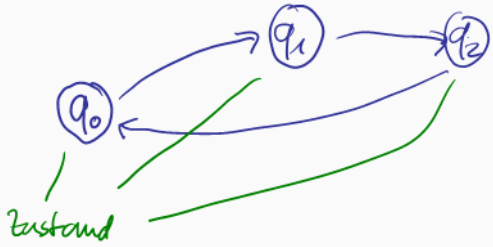
Der $*$-Operator steht für: 0 mal oder endlich oft

Σ^* = Menge aller möglichen Wörter über einem Alphabet + ε

$\Sigma^+ = \Sigma^*$ ohne ε

State machine

Ein „System" befindet sich in einem bestimmten Zustand.
Es kann in einen anderen Zustand übergehen.

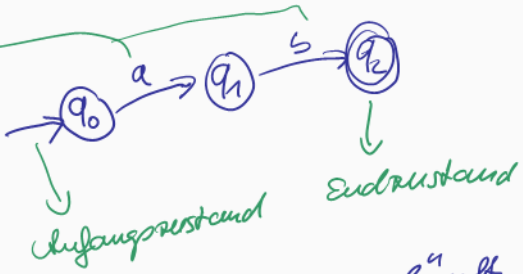


Deterministische endliche Automaten

Bestimmte State machines:

- Eingaben bestimmen Zustandsübergänge
- Zustandsübergänge sind eindeutig
- Es gibt Anfang(szustand) und End(zuständ)e

genau einen — evtl. mehrere



Wie definieren wir das formal?

- Menge der Zustände
- Eingabealphabet
- Anfangszustand
- Endzustände
- Übergangsfunktion

Def.: Deterministischer endlicher Automat

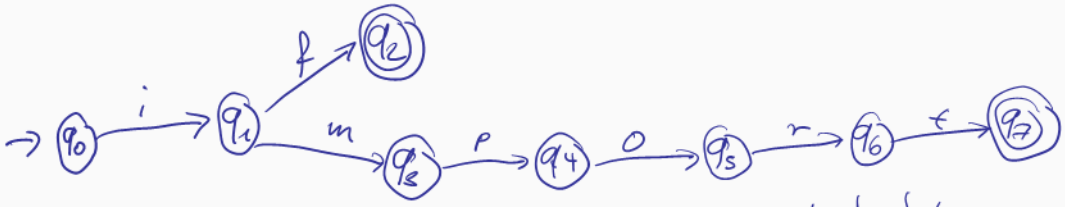
Def.: Ein **deterministischer endlicher Automat** (DFA) ist ein 5-Tupel $A = (Q, \Sigma, \delta, q_0, F)$ mit (delta)

- Q : endliche Menge von **Zuständen**
- Σ : Alphabet von **Eingabesymbolen**
- δ : die (eventuell partielle) **Übergangsfunktion** $(Q \times \Sigma) \to Q$, δ kann partiell sein
- $q_0 \in Q$: der **Startzustand**
- $F \subseteq Q$: die Menge der **Endzustände**

Parameter → Rückgabe

↳ muss nicht jede Kombination aus Zustand und Eingabezeichen definieren

Beispiel



andere Darstellung:

	i	f	m	p	o	r	t
→ q_0	q_1						
q_1		q_2	q_3				
q_2				q_4			
q_3					q_5		
q_4						q_6	
q_5							q_7
q_6							
q_7							

Eingabewörter statt Buchstaben

0 mal oder endlich oft angewendet

Eingabewort aus Σ^*

Def.: Wir definieren $\delta^* : (Q \times \Sigma^*) \to Q$: induktiv wie folgt:

- Basis: $\delta^*(q, \epsilon) = q \ \forall q \in Q$
- Induktion: $\delta^*(q, a_1, \ldots, a_n) = \delta(\delta^*(q, a_1, \ldots, a_{n-1}), a_n)$

Zustand nach Verarbeitung von $a_1 \ldots a_{n-1}$

Def.: Ein DFA akzeptiert ein Wort $w \in \Sigma^*$ genau dann, wenn $\delta^*(q_0, w) \in F$.

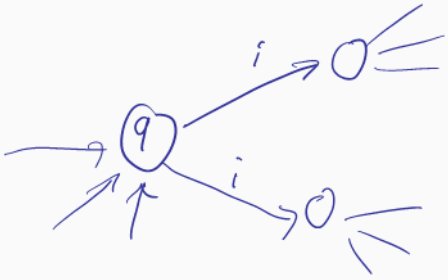
Beispiel

Der Automat von Folie 9 akzeptiert if und iinpost

Das Eingabewort ifin wird nicht akzeptiert, weil der Automat in q_2 nicht weiterkommt.

iinpo wird auch nicht akzeptiert, weil der Automat nach der Verarbeitung nicht in einem Endzustand ist.

Nichtdeterministische endliche Automaten



Es gibt für die Eingabe i keinen eindeutigen Folgezustand. Wir stellen uns vor, der Automat weiß immer, welchen Weg er gehen muss.

Def.: Nichtdeterministischer Automat

Def.: Ein **nichtdeterministischer endlicher Automat** (NFA) ist ein 5-Tupel $A = (Q, \Sigma, \delta, q_0, F)$ mit

- Q : endliche Menge von **Zuständen**
- Σ : Alphabet von **Eingabesymbolen**
- δ : die (eventuell partielle) **Übergangsfunktion** $(Q \times \Sigma) \to \mathcal{P}(Q)$
- $q_0 \in Q$: der **Startzustand**
- $F \subseteq Q$: die Menge der **Endzustände**

das muss jetzt heißen: $\mathcal{P}(Q)$, d.h. die Menge aller Teilmengen von Q.

Akzeptierte Sprachen

Def.: Sei A ein DFA oder ein NFA. Dann ist **L(A)** die von A akzeptierte Sprache, d. h.

$L(A) = \{\text{Wörter } w \mid \delta^*(q_0, w) \in F\}$

Wozu NFAs im Compilerbau?

a, b, c ? a oder b oder c

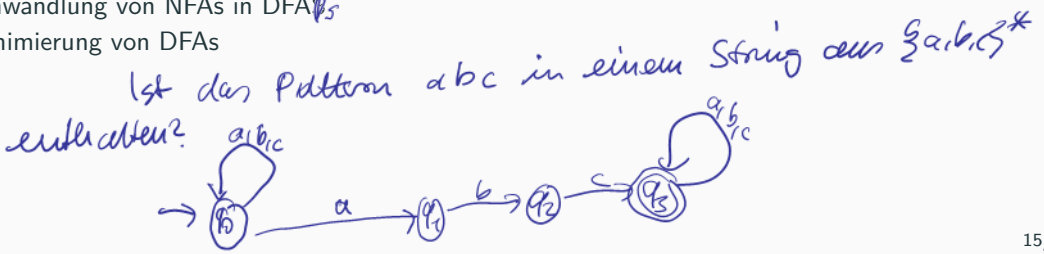
Pattern Matching (Erkennung von Schlüsselwörtern, Bezeichnern, ...) geht mit NFAs.

NFAs sind so nicht zu programmieren, aber:

Satz: Eine Sprache L wird von einem NFA akzeptiert $\Leftrightarrow$ L wird von einem DFA akzeptiert.

D. h. es existieren Algorithmen zur

- Umwandlung von NFAs in DFAs
- Minimierung von DFAs



Reguläre Sprachen

Reguläre Ausdrücke definieren Sprachen

Def.: Induktive Definition von **regulären Ausdrücken** (regex) und der von ihnen repräsentierten Sprache **L**:

- Basis:
  - ϵ und $\emptyset$ sind reguläre Ausdrücke mit $L(\epsilon) = \{\epsilon\}$, $L(\emptyset) = \emptyset$
  - Sei a ein Symbol $\Rightarrow$ a ist ein regex mit $L(a) = \{a\}$
- Induktion: Seien E, F reguläre Ausdrücke. Dann gilt:
  - $E + F$ ist ein regex und bezeichnet die Vereinigung $L(E + F) = L(E) \cup L(F)$
  - EF ist ein regex und bezeichnet die Konkatenation $L(EF) = L(E)L(F)$
  - E^* ist ein regex und bezeichnet die Kleene-Hülle $L(E^*) = (L(E))^*$
  - (E) ist ein regex mit $L((E)) = L(E)$

Vorrangregeln der Operatoren für reguläre Ausdrücke: *, Konkatenation, +

$(ab)^* = \{\epsilon, ab, abab, ababab, \ldots\}$

$(a|b)^* = \{a, b, aa, ab, ba, bb, aaa, \ldots\}$

Beispiel

$1\,(0-9)^*\,2$ beschreibt alle Wörter über $\Sigma = \{0-9\}$, die mit einer 1 beginnen, einer 2 enden und dazwischen 0 oder endlich viele Ziffern haben.

0^* : ε oder endlich viele Nullen

Wichtige Identitäten

Automaten akzeptieren Sprachen, reguläre Ausdrücke beschreiben sie.

Satz: Sei A ein DFA $\Rightarrow \exists$ regex R mit $L(A) = L(R)$.

Satz: Sei E ein regex $\Rightarrow \exists$ DFA A mit $L(E) = L(A)$.

if a < 10

Trennzeichen zwischen den Wörtern = Delimiter, hier Blank

Formale Grammatiken

Nonterminale, Variablen — Großschreibung

S → SU PR OB
SU → AR SS | PRO
PR → is | has | ...
OB → AR SS | ...
AR → the | a | an
SS → house | roof | door | ...

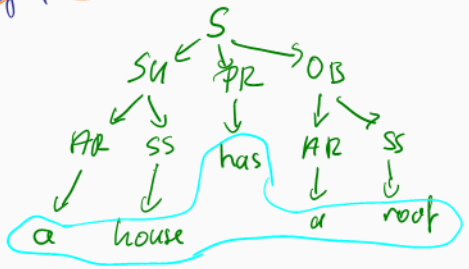
Terminale — Kleinschreibung

→ : „geht über in"

Satz → S P O

Subjekt Prädikat Objekt

SU: Subjekt
PR: Prädikat
OB: Objekt
| = oder
AR: Artikel
SS: Substantiv
PRO: Pronomen



Terminale (fertig)

Formale Definition formaler Grammatiken

Def.: Eine *formale Grammatik* ist ein 4-Tupel $G = (N, T, P, S)$ aus

- N: endliche Menge von **Nichtterminalen**
- T: endliche Menge von **Terminalen**, $N \cap T = \emptyset$
- $S \in N$: **Startsymbol**
- P: endliche Menge von **Produktionen** der Form

  $X \to Y$ mit $X \in (N \cup T)^* N (N \cup T)^*, Y \in (N \cup T)^*$

Bei unseren Regeln ist auf der linken Seite des Pfeils nur ein Nichtterminal enthalten, d.h. $(N \cup T)^*$ 0-mal vorhanden

Ableitungen

Def.: Sei $G = (N, T, P, S)$ eine Grammatik, sei $\alpha A \beta$ eine Zeichenkette über $(N \cup T)^*$ und sei $A \to \gamma$ eine Produktion von G.

Wir schreiben: $\alpha A \beta \Rightarrow \alpha \gamma \beta$ ($\alpha A \beta$ leitet $\alpha \gamma \beta$ ab).

Def.: Wir definieren die Relation $\overset{*}{\Rightarrow}$ induktiv wie folgt:

- Basis: $\forall \alpha \in (N \cup T)^* \alpha \overset{*}{\Rightarrow} \alpha$ (Jede Zeichenkette leitet sich selbst ab.)
- Induktion: Wenn $\alpha \overset{*}{\Rightarrow} \beta$ und $\beta \Rightarrow \gamma$ dann $\alpha \overset{*}{\Rightarrow} \gamma$

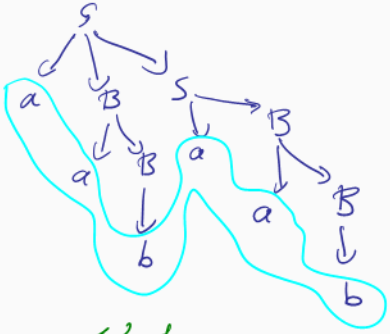
Def.: Sei $G = (N, T, P, S)$ eine formale Grammatik. Dann ist $L(G) = \{\text{Wörter } w \text{ über } T \mid S \overset{*}{\Rightarrow} w\}$ die von G erzeugte Sprache.

Beispiel

$G = (\{S, B\}, \{a, b\}, P, S)$ mit P: $S \to aBS \mid aB$
$B \to aB \mid b \mid \varepsilon$



Wort: a



Wort: aababaab

...

Reguläre Grammatiken

Def.: Eine **reguläre (oder type-3-) Grammatik** ist eine formale Grammatik mit den folgenden Einschränkungen:

- Alle Produktionen sind entweder von der Form
  - $X \to aY$ mit $X \in N, a \in T, Y \in N$ (*rechtsreguläre* Grammatik) oder
  - $X \to Ya$ mit $X \in N, a \in T, Y \in N$ (*linksreguläre* Grammatik)
- $X \to \epsilon$ ist erlaubt

Die Grammatik auf Folie 22 ist nicht regulär:
$S \to aBS \mid B \to b$ sind nicht erlaubt

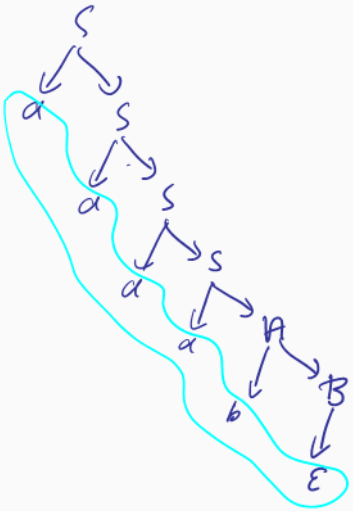
Beispiel

regulär

$S \to aS \mid aA$

$A \to bB$

$B \to \varepsilon$



Reguläre Sprachen

Satz: Die von endlichen Automaten akzeptiert Sprachklasse, die von regulären Ausdrücken beschriebene Sprachklasse und die von regulären Grammatiken erzeugte Sprachklasse sind identisch und heißen **reguläre Sprachen**.

reguläre Ausdrücke beschreiben Sprachen,
endliche Automaten (DFAs) akzeptieren Sprachen,
reguläre Grammatiken erzeugen Sprachen

Wrap-Up

Wrap-Up

- Definition und Aufgaben von Lexern
- DFAs und NFAs
- Reguläre Ausdrücke
- Reguläre Grammatiken
- Zusammenhänge zwischen diesen Mechanismen und Lexern, bzw. Lexergeneratoren



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Last modified: *0041a57 2026-09-30 theory: temporarely remove links to old annatated slides*